

DeepEE: Joint Optimization of Job Scheduling and Cooling Control for Data Center Energy Efficiency Using Deep Reinforcement Learning

Yongyi Ran*, Han Hu[†], Xin Zhou*, Yonggang Wen*

Nanyang Technological University *, Beijing Institute of Technology [†]

*{yyran, zhouxin, ygwen}@ntu.edu.sg, [†]{hhu}@bit.edu.cn

Abstract—The past decade witnessed the tremendous growth of power consumption in data centers due to the rapid development of cloud computing, big data analytics, and machine learning, etc. The prior approaches that optimize the power consumption of the information technology (IT) system and/or the cooling system always fail to capture the system dynamics or suffer from the complexity of system states and action spaces. In this paper, we propose a Deep Reinforcement Learning (DRL) based optimization framework, named DeepEE, to improve the energy efficiency for data centers by considering the IT and cooling systems concurrently. In DeepEE, we first propose a Parameterized action space based Deep Q-Network (PADQN) algorithm to solve the hybrid action space problem and jointly optimize the job scheduling for the IT system and the airflow rate adjustment for the cooling system. Then, a two-time-scale control mechanism is applied in PADQN to coordinate the IT and cooling systems more accurately and efficiently. In addition, to train and evaluate the proposed PADQN in a safe and quick way, we build a simulation platform to model the dynamics of IT workload and cooling systems simultaneously. Through extensive real-trace based simulations, we demonstrate that: 1) our algorithm can save up to 15% and 10% energy consumption in comparison with the baseline siloed and joint optimization approaches respectively; 2) our algorithm achieves more stable performance gain in terms of power consumption by adopting the parameterized action space; and 3) our algorithm leads to a better tradeoff between energy saving and service quality.

Index Terms—Deep reinforcement learning, Data center, Energy efficiency, Job scheduling, Cooling control

I. INTRODUCTION

Recent development of cloud computing, big data analytics, machine learning, and cryptocurrency has resulted in a rapid increase of electricity usage in data centers, and it is critical and urgent to reduce this energy consumption for cost saving and environmental protection. As Natural Resources Defense Council reported, U.S. data centers consumed around 91 billion kilowatt-hours of electricity in 2013 and will increase to roughly 140 billion kilowatt-hours annually by 2020, which will cost American businesses \$13 billion annually in electricity bills and emit nearly 100 million metric tons of carbon pollution per year. Typically, high energy consumption of a data center can be attributed primarily to the cooling equipment and the IT demands. The cooling system alone may account for up to 30% on average of the energy demands of a data center, while the IT system consumes around 56% of total power consumption [1]. Therefore, we can reduce both energy

costs and environmental impacts of emissions by improving energy efficiency for data centers from the perspectives of IT system and cooling system.

It is extremely challenging to achieve energy saving for data centers due to the following three aspects. First, a data center is complicated to be configured because of the dynamics of the IT and cooling systems. The IT system is responsible for executing tasks and generating heat in its computation efforts, while the cooling system is in charge of dissipating the heat. The dynamics of both systems typically include random IT workload, the heat generation, extraction and recirculation, and the time-variant ambient temperature. Failing to consider these dynamics may result in poor Power Usage Effectiveness (PUE) and high OPEX. Second, high-dimensional state space and discrete-continuous hybrid action space impose another challenge to the joint control of both systems. Hundreds and thousands of rack servers in a data center contribute a high-dimensional state space which is computationally intractable for most of the optimization methods. The action space is often discrete for the IT system to dispatch a task on a proper server, while that is continuous for the cooling system to adjust the cooling facilities (e.g., tune the airflow rate). Control of both systems should also handle this hybrid action space issue. Third, the regulation of both systems faces one additional challenge in their diverse time constants. The IT system tends to respond within seconds or even microseconds, while the cooling system (i.e., the mechanical facilities) could be in the order of minutes. This mismatched time constant may result in hot-spots, unnecessary cooling fluctuation, etc.

Siloed and joint optimization approaches [2–10] have been proposed in the literature to improve the energy efficiency of data centers. First, siloed approaches are proposed to control either IT system or cooling system alone. For the IT system control, most of the existing works perform energy saving from the perspective of workload scheduling/migrating [2, 6], which usually are model-based (e.g., queueing model for task allocation [7]) and fail to take cooling dynamics into account. For the cooling system control, most of the conventional methods employ rule-based control policies according to experts' knowledge and experience, while some control algorithms [3, 8], which aim to minimize the total cooling power, just take into account the rack temperature or room air temperature without the IT dynamics. These prevailing siloed

approaches often result in wasteful over-provisioning due to the purpose of safe operation and the lack of coordination. Second, researchers have been investigating joint IT-cooling optimization [5, 9, 10] to increase energy efficiency for data centers, where, for example, we can optimize job scheduling for the IT system and airflow rate adjustment for the cooling system concurrently. Nevertheless, most of the joint optimization approaches tend to assume some static or dynamic models for the system of interest which are sometimes either inadequate or inaccurate to capture the dynamics of various interacting processes of a data center, and also fail to solve the second and third challenges described above.

In this paper, to address the aforementioned challenges, we develop a novel and highly effective Deep Reinforcement Learning (DRL) based model-free framework, DeepEE, for energy efficient data centers to jointly optimize job scheduling and cooling control (adjusting airflow rate). In DeepEE, we employ DRL to deal with highly dynamic, time-variant and sophisticated environments such as time-varying system states and task requests, integrate parameterized action space with basic Deep Q-Network (DQN) [11] to handle the discrete-continuous hybrid action space, and apply two-time-scale control method to coordinate the diverse time constants of the IT system and cooling system. In addition, to train and validate our proposed algorithm in a safe, comprehensive and quick way, we build a simulation platform based on a commercial Computational Fluid Dynamics (CFD) software [12] to simulate the IT and cooling dynamics concurrently.

Here we summarize our main contributions as follows:

- We present DeepEE which achieves a DRL-based joint optimization for job scheduling and cooling control to improve the energy efficiency in data centers. DeepEE is model-free and does not rely on accurate and mathematically solvable system models (such as queueing models). Also, DeepEE is able to capture the dynamics and non-linear features of the IT workload and thermal process of a data center.
- In DeepEE, a Parameterized action space based Deep Q-Network (PADQN) algorithm is proposed to solve the hybrid action space problem without any approximation or relaxation. Compared to the basic DQN [11] which cannot be directly applied to solve the joint optimization problem with a discrete-continuous hybrid action space, PADQN can handle this issue well and achieve more stable performance gain in terms of power consumption.
- We apply a two-time-scale control method in DeepEE to coordinate the IT and Cooling systems with different time constants to further improve the energy efficiency.
- We build an integrated simulation platform to model IT workload and thermal dynamics concurrently based on a CFD software [12]. This platform is then used to train and validate the proposed algorithm, which can avoid operation risks compared to using a real data center environment.

The remainder of the paper is structured as follows. Related

works are reviewed in Section II. Section III describes the system models and problem formulation for jointly controlling both IT and cooling systems. In section IV, the DRL-based joint optimization algorithm is proposed to improve the energy efficiency for data centers. The evaluation results are detailed within Section V. Finally, Section VI concludes the paper.

II. RELATED WORK

In this section, we investigate the relevant research for energy efficiency in data centers. The existing work can be classified into three categories: IT system control optimization, cooling system control optimization and joint optimization of IT and cooling systems.

A. IT system control optimization

Improving the energy efficiency for the IT system can be performed from the perspectives of dynamic voltage and frequency scaling (DVFS), task scheduling/migrating and controlling the number of active servers, etc.

Tang *et al.* [13] shifted the problem of minimizing the data center cooling cost from the power space to the temperature space and defined the problem of minimizing the peak inlet temperature within a data center through task assignment, consequently leading to minimal cooling-requirement. Polverini *et al.* [7] proposed GreFar which optimizes the energy cost and fairness among distributed data centers subject to queueing delay constraints, while satisfying the maximum server inlet temperature constraints. Chavan *et al.* [14] proposed TIGER, a thermal-aware technique specifically designed to reduce the cooling cost due to storage systems in data centers. Sansottera [15] proposed a mixed integer program that aims at minimizing the total power consumption of the data center by taking into account the workload allocation and the cooling constraint. Meng *et al.* [2] considered communication-bound HPC applications and studied joint optimization of cooling and communication costs via job allocation. Many other works have been devoted to thermal-aware scheduling and resource management for data centers from different perspectives, e.g., taking response time or makespan into account. Wang *et al.* [16] proposed two workload scheduling algorithms, TASA and TASA-B, for data centers to reduce the temperatures and minimize the response time of running the workload.

Most of the existing works for IT system control optimization are model-based (e.g., queueing model for task allocation [7]) and fail to take cooling dynamics into account. In this case, in order to cool down the IT equipment for safe operation, we should have an over-provisioning on cooling power. It is not energy efficient, the cooling control needs to be taken into consideration in some cases. For example, when the average workload is in a low level for a long term, we need to turn up the supply temperature or turn down the airflow rate to avoid over-provisioning on cooling power, while only scheduling the workload cannot achieve this.

B. Cooling system control optimization

Cooling control is essential and critical to optimize the energy consumption in a data center, which can be performed

from the perspective of fan speed and CRAC unit configuration.

One method is to control the fan speeds to reduce cooling power while maintaining the temperature in the data center [17]. More works have been devoted to CRAC unit configuration. Zhou *et al.* [8] proposed a unified framework with Model Predictive Controller (MPC) to minimize the cooling power by adjusting the supply air temperature, tile utilization and variable frequency drive while meeting the thermal requirements. Beghi *et al.* [3] developed a model-based approach for the design of an efficient control strategy for CRAC systems. The problem is formulated as a non-linear constrained optimization problem and it is solved by particle swarm optimization.

These approaches for cooling system control optimization always just take into account the rack temperature or room air temperature without considering the IT dynamics. Such siloed approaches often result in wasteful over-provisioning due to the purpose of safe operation and the lack of coordination/joint control.

C. Joint optimization of IT and cooling systems

To achieve better energy efficiency for data centers, a joint optimization of IT and cooling control is more promising and highly desired which can be performed by concurrently dispatching job/workload and configuring cooling system (e.g., supply temperature, airflow rate).

Pakbaznia *et al.* [18] presented an integer linear programming for the server consolidation and the task assignment to minimize the data center server and cooling power costs. An optimum supplied cold air temperature is chosen to reduce the cooling power, whereas the server power is decreased by properly dispatching the arrival tasks to different servers and setting an appropriate voltage-frequency level for each server according to the types of running tasks. Parolini *et al.* [9] presented a coordinated control strategy from a cyberphysical system perspective to minimize the total power consumption in the data centre with respect to both QoS and energy efficiency. Wan *et al.* [5] considered a cross-layer approach to minimize the holistic energy consumption in data centers by formulating a mixed integer nonlinear programming and developing an efficient JOINT algorithm to solve it. JOINT reduces the energy consumption by collaborating different system components across different control layers, and dynamically sets system parameters depending on the current workload. Sun *et al.* [6] proposed an online scheduling algorithm for HPC applications to minimize the makespan subject to a constraint on thermal. This algorithm performs both job allocation and thermal management by leveraging the novel notion of thermal-aware load.

Most of the existing works suffer from one or more challenges as discussed in Section I. In this paper, to tackle those challenges, we 1) propose a DRL-based framework, named DeepEE, to jointly optimize task scheduling and cooling control, 2) employ parameterized action space techniques to handle the discrete-continuous action space problem, and 3)

introduce a two-time-scale control mechanism to coordinate the IT and cooling systems more efficiently and accurately.

III. PROBLEM FORMULATION

In this section, we first present the system architecture of DeepEE for the joint optimization of IT and cooling systems based on DRL. Then, we describe the system models. Finally, the problem formulation is given.

A. System Architecture

As illustrated in Fig. 1, a typical data center comprises IT system and cooling system. The system architecture of DeepEE, which follows the general agent-environment interaction model (of both traditional RL and the emerging DRL), consists of a DRL agent and the data center environment.

1) *IT system*: The IT system of a data center contains a suit of facilities (e.g., racks and servers) and associated management softwares (e.g., task scheduler). As shown in Fig. 1(a), a data center typically has several rows of server racks, and each rack contains a set of servers. Each server could have single-core or multi-core processors. All servers in a rack share the same power unit. The main purpose of the IT system in a data center is to provide user-satisfactory services, e.g., high-performance computing. Thus it is essential to outfit management software to achieve this purpose. The commonly used management software in commercially data centers follows a conventional task queuing and issuing paradigm [19]. When a user submits a new task to the data center, the Task Scheduler will store the newly arrived task into an Arrival Queue, and then the queued tasks will be dispatched to a proper server according to the job scheduling policy.

2) *Cooling system*: The cooling system is equipped with cooling equipment to dissipate the heat generated by the active servers inside a data center. As illustrated in Fig. 1(b), we consider the data center in this paper is an air-cooled data center with raised floor supply and ceiling return structure. Cold aisle containment technique is employed to prevent recirculation effect. The main cooling equipment is Precision Cooling Unit (PCU). The cold air flow (blue arrows) supplied by a PCU passes across the raised floor plenum and comes through the perforated tiles (in the floor underneath the cold aisles) to the enclosed cold aisle. Then the cold air will be pumped into the rack by chassis fans and carry away the heat generated by rack servers. Finally, the hot air will be exhausted toward the hot aisles and extracted from the data centre room by the PCUs and moved to the outdoor environment.

3) *DRL Agent*: As shown in Fig. 1(c), in the DRL agent, we design and implement DRL based algorithms to provide control policies for the IT system (e.g., dispatching tasks) and cooling system (e.g., adjusting airflow rate). The agent should be trained first when the system begins to run. The training is actually to construct one or more Deep Neural Networks (DNNs). Typically, we should train a DNN called Deep Q-Network (DQN) for deep Q-learning [11] to derive the correlation between each state-action pair (s, a) and the corresponding value function $Q(s, a)$, which indicates the

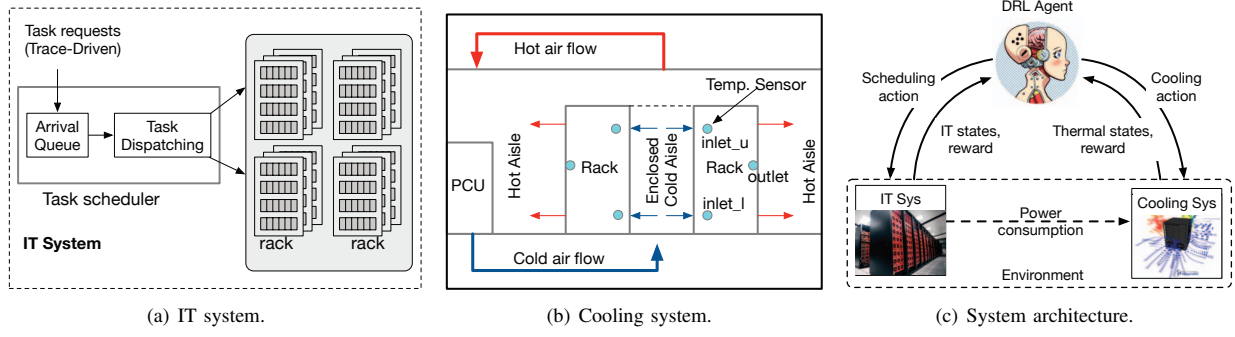


Fig. 1: Overview of DeepEE for Data Centers. (a) IT system is to provide user-satisfactory services with a suit of facilities (e.g., racks and servers) and associated management softwares (e.g., task scheduler). (b) Cooling system is equipped with cooling equipment (e.g., PCU) to dissipate the heat. (c) System architecture illustrates the interaction between DRL agent and the environment. The DRL agent is to provide control policies for IT system (e.g., dispatching tasks) and cooling system (e.g., adjusting airflow rate). The environment returns the states of the data center and rewards to DRL agent.

expected accumulative rewards to be maximized. After the agent is well-trained, the DRL agent can be used to provide optimal control policies for the IT system and cooling system.

4) *Workflow*: The workflow of DeepEE is illustrated in Fig. 1(c). The interaction between the DRL agent and the data center environment (including the IT system and cooling system) is a continuous process. The DRL agent needs to make a decision a_t according to the current state s_t of the data center at each decision epoch t . Then the decision a_t would be received by the data center and the corresponding actions would be executed. After that, the state would be updated (new state s_{t+1}) and s_{t+1} would be transferred to the DRL agent for making future decisions. An instant reward r_t from the environment also would be provided to the DRL agent.

B. System Models

In this subsection, we present the system models including workload model, task model, and thermal model respectively in a data center.

1) *Task Model*: In this paper, we target compute-intensive tasks for high performance computing (HPC) data centers. We consider that a data center consists of N server racks. In each server rack n , there are M_n servers, and each server has c_{total} CPU cores. Here we consider a homogeneous hardware environment, i.e., all racks comprise the same number of servers, and each server has the same computing capability and power performance.

A task submission usually contains [20]: 1) the executable files, 2) the input data, 3) the required number of CPU cores and the estimated runtime, and 4) other requirements such as priority. Given that the job run-times are normally overestimated by the users [21], we treat the number of required CPU cores as the main factor for allocating a compute-intensive task to a certain server. All the newly arrived tasks will be pushed into the Arrival Queue (as illustrated in Fig. 1(a)), and then Task Scheduler selects a candidate task according to First-Come-First-Serve (FCFS) which will be dispatched

at the next decision epoch. The candidate task is the task in the Arrival Queue which is selected to be dispatched at the next epoch. For simplicity, we assume that the Task Scheduler only allocate one task at one time. Given a candidate task i , let c_i be the number of required CPU cores. When the Task Scheduler dispatches the task to a certain server k , the following constraint should be satisfied:

$$c_i \leq c_k^a. \quad (1)$$

It should be noted that 1) real workload traces from [22] will be used to generate the tasks and drive the experiments in this paper; 2) if the number of required CPU cores of a task exceeds the total number of CPU cores c_{total} , this task will be divided into two or more sub-tasks.

2) *Workload Model*: To track the workload states of the IT system, three basic variables are kept for each server k : the number of available CPUs c_k^a (where $c_k^a \leq c_{total}$), the utilization u_k and the power consumption p_k . c_k^a and u_k can be easily and intuitively updated when a new task is allocated to a server k . The power consumption of a server can be estimated using the following linear power model [18, 23]:

$$p_k = P_{idle} + (P_{full} - P_{idle}) * u_k, \quad (2)$$

where P_{idle} and P_{full} indicate the power consumption by a server at the idle state and fully utilized state, respectively. It has been proved that the linear power model is very useful and accurate in estimating the power consumption for rack servers [24]. The total power consumption of IT system can be defined as:

$$P_{it} = \sum_{n=1}^N \sum_{k=1}^{M_n} p_k. \quad (3)$$

Note that although we use linear model to estimate the power consumption for each server, our DRL-based algorithm does not depend on this specific power consumption model.

Here we use $\mathbf{c}_a = (c_1^a, \dots, c_K^a)$, $\mathbf{u} = (u_1, \dots, u_K)$ and $\mathbf{p} = (p_1, \dots, p_K)$ ($K = \sum_{n=1}^N M_n$) to denote the number available

CPU cores, the utilization, and the power consumption for all servers respectively. Then, the state of the IT system can be represented as

$$\mathbf{s}_{it} = (\mathbf{c}_a, \mathbf{u}, \mathbf{p}). \quad (4)$$

3) *Thermal Model*: In order to capture the thermal dynamics of a data center, we sample three temperature data for each rack periodically via temperature sensors. As illustrated in Fig. 1(b), we deploy two temperature sensors in front of each rack n (position: upper and lower) to derive two inlet temperatures $(T_{in,u}^n, T_{in,l}^n)$, and deploy one temperature sensor in the back (position: middle) to derive the outlet temperature T_o^n . Here we use $\mathbf{T}_{in,u} = (T_{in,u}^1, \dots, T_{in,u}^N)$, $\mathbf{T}_{in,l} = (T_{in,l}^1, \dots, T_{in,l}^N)$ and $\mathbf{T}_o = (T_o^1, \dots, T_o^N)$ to represent the inlet temperatures and the outlet temperature of all racks respectively. Then the thermal states of a data center can be given as

$$\mathbf{s}_{thl} = (\mathbf{T}_{in,u}, \mathbf{T}_{in,l}, \mathbf{T}_o). \quad (5)$$

As shown in Fig. 1(b), PCUs are equipped to cool the data center. Assume that there are J (e.g., $J = 3$) PCUs. In this paper, the supply temperature of each PCU is set as a constant (e.g., 19°C), while the airflow rates $\mathbf{f} = (f^1, f^2, \dots, f^J)$ of these PCUs can be adjusted to control the cooling power. The airflow rate cannot exceed the maximum value that can be supplied by the PCUs, i.e.,

$$0 < f^j \leq f_{max} \quad (j = 1, 2, \dots, J). \quad (6)$$

In addition, let P_{cool} be the total power consumption of the cooling equipment PCUs which can be measured via power meters [25].

C. Problem Statement

We describe the joint optimization of task scheduling and cooling control problem in this section.

As illustrated in Fig. 1(c), given the workload states and the thermal states, the joint optimization problem involves two decisions. First, a candidate task in the Arrival Queue should be assigned to a proper server k , where k is the index (integer) of a server and $k \in \{0, 1, \dots, \sum_{n=1}^N M_n\}$. If $k = 0$, it means that this candidate task would not be assigned to any server at this epoch. Second, the DRL agent should set an optimal airflow rate for the PCUs according to the current states of the data center. For simplicity, DRL agent can decide whether or not the airflow rate of each PCU should be increased or reduced. Here we use $\mathbf{x} = (x^1, x^2, \dots, x^J)$ to denote this decision, where $-x_0 \leq \mathbf{x} \leq x_0$ and x_0 is an action bound. Note that $-x_0 \leq \mathbf{x} \leq x_0$ means $-x_0 \leq x^j \leq x_0 (j = 1, 2, \dots, J)$.

The joint optimization problem aims to minimize the power usage effectiveness (PUE) while preventing overheating in the rack server and keeping the load balance. Specifically, the objective function consists of three aspects:

- 1) $PUE = (P_{it} + P_{cool})/P_{it}$, which should be minimized.
- 2) $\ln(1 + \exp(T_o^n - \psi_T))$, the penalty of the overheating for each rack n , where ψ_T indicates the threshold of the outlet temperature of a rack.

- 3) $\ln(1 + \exp(u_k - \psi_u))$ the penalty of overload for each server k , where ψ_u indicates the threshold of the utilization of a server.

So the optimization problem can be expressed as:

$$\begin{aligned} \min \quad & \Gamma \triangleq PUE + \beta_1 \frac{1}{N} \sum_{n=1}^N \ln(1 + \exp(T_o^n - \psi_T)) \\ & + \beta_2 \frac{1}{N} \sum_{n=1}^N \frac{1}{M_n} \sum_{k=1}^{M_n} \ln(1 + \exp(u_k - \psi_u)) \\ \text{s.t.} \quad & (1), (6) \end{aligned} \quad (7)$$

where β_1 and β_2 are the weight factors for the penalty of overheating and overload respectively, which imply the importance of overheating and overload.

In this objective function, we use “softplus” function for the penalty, which is also applied in [4, 26, 27]. “softplus” function is enticingly smooth and differentiable. It is much easier and more efficient than the straightforward penalty functions $\max(0, T_o^n - \psi_T)$ and $\max(0, u_k - \psi_u)$ for the cost optimization.

IV. PROPOSED DRL-BASED JOINT OPTIMIZATION ALGORITHM

In this section, we present the proposed “PADQN” algorithm for our DeepEE framework to address the challenges described in Section I. First, we combines DQN with parameterized action space to jointly control the IT and cooling systems. Then, we introduce two-time-scale control in DeepEE to coordinate both systems more efficiently.

A. DRL for Discrete and Continuous Action Space

We present some background knowledge about reinforcement learning (RL) and deep reinforcement learning (DRL).

1) *Discrete Action Space*. In general, the traditional Q-learning and DQN-based DRL algorithms only work for the control problems which have a discrete action space. For the traditional Q-learning that is based on Bellman Equation, the learning phase is to derive a Q-table which reflects the correlation between each state-action pair $(\mathbf{s}_t, \mathbf{a}_t)$ of the system and its action-value function $Q(\mathbf{s}_t, \mathbf{a}_t)$, which is the expected discounted cumulative reward:

$$Q(\mathbf{s}_t, \mathbf{a}_t) = \mathbb{E}[R_t | \mathbf{s}_t, \mathbf{a}_t], \quad (8)$$

where $R_t = \sum_{k=t}^T \gamma^k r(\mathbf{s}_k, \mathbf{a}_k)$ and $\gamma \in [0, 1]$ is the discount factor. A commonly used off-policy algorithm takes the greedy policy: $\pi(\mathbf{s}_t) = \arg \max_{\mathbf{a}_t} Q(\mathbf{s}_t, \mathbf{a}_t)$.

DeepMind [11] presented DRL that extends Q-learning with a Deep Q-Network (DQN), i.e., a neural network $Q(\mathbf{s}, \mathbf{a}; \theta)$ is applied to approximate the optimal Q^* , where θ is the network weights. In the t -th iteration, the DQN updates its parameters via the gradient of the least squares loss function

$$L_t(\theta) = [y_t - Q(\mathbf{s}_t, \mathbf{a}_t; \theta)]^2. \quad (9)$$

y_t is the target value that can be estimated by:

$$y_t = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \max_{\mathbf{a}'} Q(\mathbf{s}_{t+1}, \mathbf{a}'; \theta^-), \quad (10)$$

where θ^- is the weight parameters of the target network.

In practice, two effective techniques [11], i.e., experience replay and target network, are applied for the DQN training. Unlike the traditional Q-learning, the DRL agent updates the DQN with a mini-batch from an experience replay buffer, which stores state transition samples collected during interacting with the environment. Experience replay can smooth out the learning and avoid oscillations or divergence. On the other hand, the DRL agent uses a target network to estimate the target value y_t . The parameters θ^- are slowly updated with the parameters of DQN θ . The update rate can be controlled by a parameter τ , i.e., $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$.

2) *Continuous Action Space*. To handle the continuous control problems, policy-based methods are usually employed. In [28], DeepMind extended the traditional actor-critic approach by applying DNN, named Deep Deterministic Policy Gradient (DDPG), for continuous control. DDPG maintains a parameterized actor function $\pi(s_t; \omega)$ which can be implemented using a DNN and a parameterized critic function $Q(s_t, a_t; \theta)$ which can be implemented by employing a DQN. $\pi(s_t; \omega)$ outputs the current policy by mapping a state to a specific action, while $Q(s_t, a_t; \theta)$ returns Q value for a given state-action pair. According to [28], the actor network can be updated by applying chain rule to the expected cumulative reward J with respect to the actor parameters ω :

$$\nabla_{\omega} J = \mathbb{E}[\nabla_{\omega} \pi(s; \omega)|_{s=s_t} \nabla_a Q(s, a; \theta)|_{a=\pi(s_t; \omega), s=s_t}] \quad (11)$$

B. Parameterized Action Space DQN (PADQN)

To tackle the first two challenges described in Section I and fit the existing DRL techniques into the hybrid action space without any approximation, we apply the parameterized action space MDP (PAMDP) [29] into DQN-based DRL algorithm, called Parameterized Action Space DQN (PADQN), to jointly optimize job scheduling and cooling control in a data center.

In order to apply DRL (no matter which method to use), here we define the state space, action space and reward function of the DRL-based framework as follows:

- **State Space:** The state consists of the following parts: the number of required CPU cores c_i of the candidate task i , the airflow rates $\mathbf{f}$, workload states $\mathbf{s}_{it}$ and thermal states $\mathbf{s}_{thl}$. Formally, the state vector is represented as $\mathbf{s} = (c_i, \mathbf{f}, \mathbf{s}_{it}, \mathbf{s}_{thl})$.
- **Action Space:** As discussed before, the action for task scheduling is $k \in \{0, 1, \dots, \sum_{n=1}^N M_n\}$, where k is an integer indicating the index of the selected server. The action for adjusting airflow rate is $\mathbf{x} = (x^1, x^2, \dots, x^J)$ as described in Section III-C. Formally, the action vector can be expressed as $\mathbf{a} = (k, \mathbf{x})$.
- **Reward:** We define the immediate reward that the DRL agent receives as $r = r_0 - \Gamma - \beta_3$, where r_0 is a large constant which can ensure that the reward is a positive value, Γ can be calculated by Equation (7) and β_3 is a penalty for invalid actions (i.e., violate the constraints in (6) and (1)).

By applying the parameterized action space MDP (PAMDP) [29] into DQN-based DRL algorithm, called Parameterized

Action Space DQN (PADQN), the hybrid action space of the joint optimization problem can be written as a parameterized action space:

$$\mathcal{A} = \{(k, \mathbf{x}_k) | k \in \{0, 1, \dots, \sum_{n=1}^N M_n\}, -x_0 \leq \mathbf{x}_k \leq x_0\}, \quad (12)$$

where k is the discrete action for the task scheduling while $\mathbf{x}_k$ is a continuous action for adjusting airflow rate associated with the action k . For convenience, we define $\mathcal{K} = \{0, 1, \dots, \sum_{n=1}^N M_n\}$ and $\mathcal{X} = \{\mathbf{x}_k | -x_0 \leq \mathbf{x}_k \leq x_0\}$. With this parameterized action space $\mathcal{A}$, the action value function can be expressed as: $Q(\mathbf{s}, \mathbf{a}) = Q(\mathbf{s}, k, \mathbf{x}_k)$. If the system state is $\mathbf{s}_t$ and follows the action $\mathbf{a}_t = (k_t, \mathbf{x}_{k_t})$ at decision epoch t , then the Bellman equation can be written as

$$Q(\mathbf{s}_t, k_t, \mathbf{x}_{k_t}) = \mathbb{E}[r_t + \gamma \max_{k \in \mathcal{K}} \sup_{\mathbf{x}_k \in \mathcal{X}} Q(\mathbf{s}_{t+1}, k, \mathbf{x}_k) | \mathbf{s}_t = \mathbf{s}] \quad (13)$$

For the expression “maxsup” in the Equation (13), we can first derive $\mathbf{x}_k^* = \arg \sup_{\mathbf{x}_k \in \mathcal{X}} Q(\mathbf{s}_{t+1}, k, \mathbf{x}_k)$ for each $k \in \mathcal{K}$, and then pick the largest $Q(\mathbf{s}_{t+1}, k, \mathbf{x}_k^*)$. However, it is difficult to calculate the supremum over continuous space $\mathcal{X}$. For a given k and Q function, we can find that

$$\mathbf{x}_k^Q(\mathbf{s}) = \arg \sup_{\mathbf{x}_k \in \mathcal{X}} Q(\mathbf{s}, k, \mathbf{x}_k) \quad (14)$$

is a function of the state $\mathbf{s}$. Therefore, referring to DQN, we can use a deep neural network $Q(\mathbf{s}, k, \mathbf{x}_k; \theta)$ to approximate $Q(\mathbf{s}, k, \mathbf{x}_k)$, where θ is the network parameters. For such a $Q(\mathbf{s}, k, \mathbf{x}_k; \theta)$, we can use a deterministic policy network $\mathbf{x}_k^Q(\mathbf{s}; \omega)$ to approximate $\mathbf{x}_k^Q$ in (14), where ω represents the network parameters. The output of the policy network $\mathbf{x}_k^Q(\mathbf{s}; \omega)$ is a continuous parameter (i.e., the continuous action for adjusting the airflow rate in this paper).

In this paper, we just use one neural network $\mathbf{x}(\mathbf{s}; \omega)$ to approximate all $\mathbf{x}_k^Q$ (for all $k \in \mathcal{K}$). In our joint optimization framework, 1) all continuous actions $\mathbf{x}_k$ ($k \in \mathcal{K}$) have the same continuous action space with each other and have the same control target – airflow rate; 2) a task allocation will not affect the ambient temperature in a very short time due to the thermal lag; 3) the cooling system will not react to a task allocation immediately due to the two-time-scale control (will be described in the next section). Therefore, we only use one neural network $\mathbf{x}(\mathbf{s}; \omega)$ to generate the continuous cooling control without regard to the task scheduling action k . However, the output of $\mathbf{x}(\mathbf{s}; \omega)$ as well as the system states will be fed into the Q-network $Q(\mathbf{s}, k, \mathbf{x}_k; \theta)$ which outputs $|\mathcal{K}|$ action values. Then the DRL agent picks up an action with the maximum action value for task scheduling.

Similar to the training of DQN, we can update the network parameters θ of $Q(\mathbf{s}, k, \mathbf{x}_k; \theta)$ by using the gradient of the least squares loss function in the t -th iteration,

$$L_t^Q(\theta) = [y_t - Q(\mathbf{s}_t, k_t, \mathbf{x}_{k_t}; \theta)]^2, \quad (15)$$

where y_t is the target value that can be defined by:

$$y_t = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \max_{k \in \mathcal{K}} Q(\mathbf{s}_{t+1}, k, \mathbf{x}_k(\mathbf{s}_{t+1}; \omega_t); \theta^-), \quad (16)$$

where θ^- are the parameters of the target network $Q' = Q(s, k, \mathbf{x}_k; \theta^-)$. Therefore, the gradients for updating θ [11] can be calculated by

$$\nabla_{\theta} L_t^Q(\theta) = \{y_t - Q(s_t, k_t, \mathbf{x}_{k_t}; \theta)\} \nabla_{\theta} Q(s_t, k_t, \mathbf{x}_{k_t}; \theta). \quad (17)$$

$\mathbf{x}_k(s; \omega)$ is a deterministic policy network, thus the network parameters ω can be updated like the Equation (11):

$$\begin{aligned} \nabla_{\omega} J = \\ \mathbb{E}[\nabla_{\omega} \mathbf{x}_k(s; \omega)|_{s=s_t} \nabla_{\mathbf{x}} Q(s, k, \mathbf{x}_k(s; \omega); \theta)|_{\mathbf{x}=\mathbf{x}_k(s_t; \omega), s=s_t}] \end{aligned} \quad (18)$$

C. Two-Time-Scale Control

In order to handle the third challenge described in Section I, we introduce a two-time-scale control mechanism into the proposed PADQN.

Given the aforementioned discussion, the proposed algorithm, which is to jointly optimize task scheduling and cooling control for energy efficiency of a data center, should have the following features: 1) talking with each other when making a decision, 2) using two different time scales to control these two systems respectively. For the first feature, in our proposed PADQN, the output of the policy network $\mathbf{x}_k(s; \omega)$ (i.e. the continuous action for adjusting airflow rate) will be fed into the Q-network $Q(s, k, \mathbf{x}_k; \theta)$ which will then output $|\mathcal{K}|$ action values and the DRL agent picks up an action with the maximum action value for task scheduling. In addition, both the workload states and the thermal states are used as input for these two neural networks. For the second feature, we tend to make a decision at every decision epoch for the IT system and at every t_{cool} decision epochs for the cooling system. Here we introduce a time factor t_f to indicate the system time state. If the decision epoch t equals the integer times of t_{cool} , $t_f = 1$, otherwise, $t_f = -1$, i.e.,

$$t_f = \begin{cases} 1 & t = d \times t_{cool}, d \in \mathcal{R}, \\ -1 & \text{otherwise.} \end{cases} \quad (19)$$

Then, t_f is used as an input for the policy network $\mathbf{x}_k(s; \omega)$ and the Q-network $Q(s, k, \mathbf{x}_k; \theta)$. That means we can extend the state vector in Section IV-B as $\mathbf{s} = (c_i, t_f, \mathbf{f}, \mathbf{s}_{it}, \mathbf{s}_{th})$. In the environment side, in order to drive the DRL agent to make an optimum decision for the cooling system at a right epoch, a penalty (the item β_3 in the reward function in IV-B) will be added to the reward if $t_f = -1$ and the cooling action $\mathbf{x} \notin \{\mathbf{z} | -x_{tf} < \mathbf{z} < x_{tf}\}$, where x_{tf} is a threshold for bounding the valid cooling action.

Finally, according to the description above, the proposed algorithm PADQN for our DeepEE framework can be summarized in Algorithm 1.

V. EVALUATION

In this section, we briefly introduce our simulation platform, carry out extensive simulation experiments on this platform to evaluate our proposed algorithm, and present and analyze the simulation results.

Algorithm 1 PADQN Algorithm for DeepEE

Input: Randomly initialize the Q-network $Q(s, k, \mathbf{x}_k; \theta)$ and policy network $\mathbf{x}_k(s; \omega)$ with weights θ and ω respectively. Initialize target network Q' with weights $\theta^- \leftarrow \theta$. Initialize replay buffer $\mathcal{R}$, exploration parameter ϵ , minibatch size B .

```

1: Receive initial observation state  $\mathbf{s}_1$ 
2: for  $t=1$  to  $T$  do
3:   With probability  $\epsilon$ , select a random action  $\mathbf{a}_t = (k_t, \mathbf{x}_{k_t})$ 
4:   from distribution  $\mu$ , otherwise
5:   1) select the continuous action  $\mathbf{x}_{k_t} \leftarrow \mathbf{x}_k(s_t; \omega)$ 
6:   2) select  $k_t$  such that  $k_t = \arg \max_{k \in \mathcal{K}} Q(s_t, k, \mathbf{x}_k; \theta)$ 
7:   Take action  $\mathbf{a}_t$ , observe reward  $r_t$  and the next state  $\mathbf{s}_{t+1}$ 
8:   Store the transition  $\{\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}\}$  into  $\mathcal{R}$ 
9:   Sample a random minibatch of  $B$  transitions
10:   $\{\mathbf{s}_j, \mathbf{a}_j, r_j, \mathbf{s}_{j+1}\}_{1 \leq j \leq B}$ 
11:  Set  $y_t = r(s_t, \mathbf{a}_t) + \gamma \max_{k \in \mathcal{K}} Q(s_{t+1}, k, \mathbf{x}_k(s_{t+1}; \omega_t); \theta^-)$ 
12:  Compute the stochastic gradient  $\nabla_{\theta} L_t^Q(\theta)$  and  $\nabla_{\omega} J$ 
13:  according to (17) and (18) respectively.
14:  Update the parameters for the network  $Q(s, k, \mathbf{x}_k; \theta)$ 
15:  and  $\mathbf{x}_k(s; \omega)$ 
16:  Update the target  $Q$  network:
17:   $\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$ 
18: end for

```

A. Setup

In order to carry out experiments and evaluate our proposed algorithm, we build an integrated simulation platform to model and simulate the IT and cooling systems of a data center. As illustrated in Fig. 1, our simulation platform contains three main parts: IT system, cooling system and DRL agent.

For the IT system, we implement the modules shown in Fig. 1(a) in Matlab to simulate the task scheduling and workload dynamics. In this paper, we refer to the configuration of a real data center at National Supercomputing Centre (NSCC) Singapore to model the data center for simulation. There are total 26 racks including 20 supercomputing racks and 6 management racks. In our experiments, we use the supercomputing racks as the targets, i.e., $N = 20$. There are 42 1U servers for each supercomputing rack and 24 CPU cores for each server (i.e., $M_n = 42$ and $C_{total} = 24$). To drive the task scheduling in IT part, we use a real workload trace named “LLNL Thunder” from [22] to generate task requests. We extracted total 14,000 tasks: 10,000 tasks are used for training, 2,000 tasks for initializing the simulation platform and the remaining 2,000 tasks for evaluating. A task that requires more than 24 CPU cores will be partitioned into several sub-tasks, where each sub-task requires up to 8 CPU cores. In addition, we set $P_{idel} = 100W$, $P_{full} = 300W$ for each server.

For the cooling system, we firstly use a commercial CFD software [12] to construct a 3D CFD model for a data center based on the basic infrastructure information (e.g., the layout of the data center, the server specification sheet, PCU configuration, etc.) of a real data center at National Supercomputing Centre (NSCC) Singapore (Fig. 2). In this model, 3 PCUs are employed to provide cooling air for this data center, i.e., $J = 3$. The initial airflow rate is 50% of the rated airflow rate, and the airflow control range is $-10\% \leq \mathbf{x} \leq 10\%$, i.e., the cooling action bound x_0 is 10% of the rated airflow rate. Then,

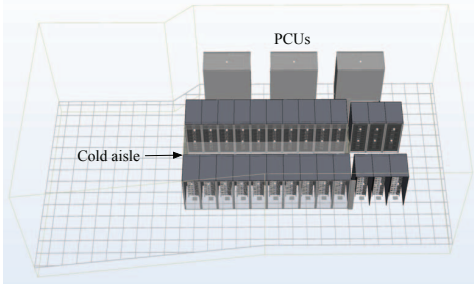


Fig. 2: 3D CFD model. We build this 3D model based on the basic infrastructure information (e.g. the layout of the data center, the server specification sheet, PCU configuration, etc.) of a real data center at National Supercomputing Centre (NSCC) Singapore.

the CFD model is used to simulate the temperature dynamics s_{thl} and output the total power consumption of the cooling equipment P_{cool} after finishing once simulation.

For the DRL Agent, we implement our proposed PADQN algorithm using “Tensorflow” to provide control policies for the IT and cooling systems. For $Q(s, k, \mathbf{x}_k; \theta)$, we used a 2-layer fully-connected feedforward neural network, which includes 512 and 64 neurons in the first and second layer respectively and utilized the ReLU as the activation function. While for $\mathbf{x}_k(s_t; \omega)$, we employed a 3-layer fully-connected feedforward neural network, with 512, 1024 and 32 neurons in the first, second and third layer respectively. In the final output layer, the “tanh” is used as activation function to ensure output values fall into a symmetric interval. In addition, some other key parameters are set as: minibatch size $B = 64$, discount factor $\gamma = 0.99$, initial learning rate $= 0.00025$, $\tau = 0.01$, the decision-making interval for IT system is $d = 10$ seconds and that for cooling equipment is 5 minutes, i.e., $t_{cool} = 30$, $\psi_T = 30^\circ\text{C}$, $\psi_u = 90\%$, $x_{tf} = 1\%$ and $\beta_1 = \beta_2 = 1$.

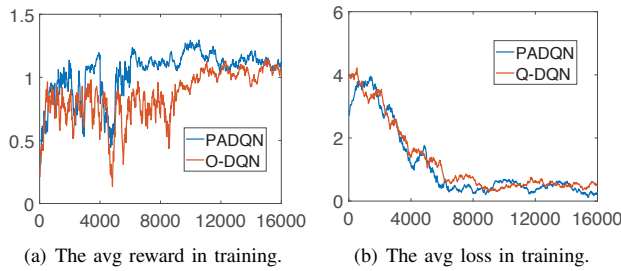


Fig. 3: The training results for PADQN and O-DQN. PADQN begins to become converged after around 6,000 epochs, while O-DQN begins to converge after about 10000 epochs.

B. Performance Metrics

In order to evaluate the performance of the proposed algorithm, we define the following performance metrics:

- Average reward: the average reward returned from the environment. The reward is defined in Section IV-B.

- Average task waiting time: the average sojourn time that a task is blocked in the Arrival Queue.
- Average PUE: this metric is defined in Section III-C.
- The average number of hot-spots: a hot-spot means the outlet temperature of a rack exceeds the threshold (30°C). Here we count the time-average number of hot-spots.
- The total completion time: the time that the system takes to finish all the queued tasks.

C. Baseline Algorithms

We compare our proposed PADQN algorithm with four baseline solutions:

- IT Control Optimization Algorithm (ICO) [15]: This algorithm formulates a mixed integer program that aims at minimizing the total power consumption of the data center by only taking into account dispatching the IT workload. The cooling capacity can not be adjusted and only be used as a constraint.
- Cooling Control Optimization Algorithm (CCO) [8]: This algorithm employs a unified framework with Model Predictive Controller (MPC) to minimize the cooling power over the CRAC units by controlling the rate of changes of supply air temperature, variable frequency drive and tile utilization. In order to be consistent with this paper, we make a minor modification that only the airflow rate will be adjusted.
- Joint IT and Cooling Control Optimization Algorithm (JCO) [6]: This joint optimization algorithm is to minimize the makespan for a set of HPC applications by performing both job assignment and thermal management. For consistency, we modify the algorithm in [6] to control the airflow rate to achieve thermal management instead of adjusting dynamic voltage and frequency scaling (DVFS).
- Original DQN (O-DQN): We discretize the continuous action space of the cooling system $\mathcal{X} = \{x | -x_0 \leq x \leq x_0\}$ into a discrete set $\mathcal{X}' = \{-x_0, -x_0 + 1\%, \dots, x_0 - 1\%, x_0\}$, and then DQN based DRL algorithm [11] can be used to solve the joint optimization problem. In general, the division of the action space is finer, the control will be more precise. However, this discretization will largely increase the dimension of the discrete action space. Take into account the practical operation, here we choose 1% as the control step size.

D. Results of Training PADQN

In this section, we present the results of training our proposed PADQN and O-DQN. We run the simulation for 16,000 epochs (iterations) to train both algorithms. The time step d is 10 seconds (the time interval between two decision epochs). We sampled an average reward and loss every 20 epochs. The learning curves of PADQN and O-DQN are shown in Fig. 3. It can be observed from Fig. 3 that PADQN can (within just 6000 decision epochs) reaches a better solution (that derives a high reward) more quickly than O-DQN. PADQN begins to become converged after around 6,000 epochs, while O-DQN begins to converge after about 10000 epochs.

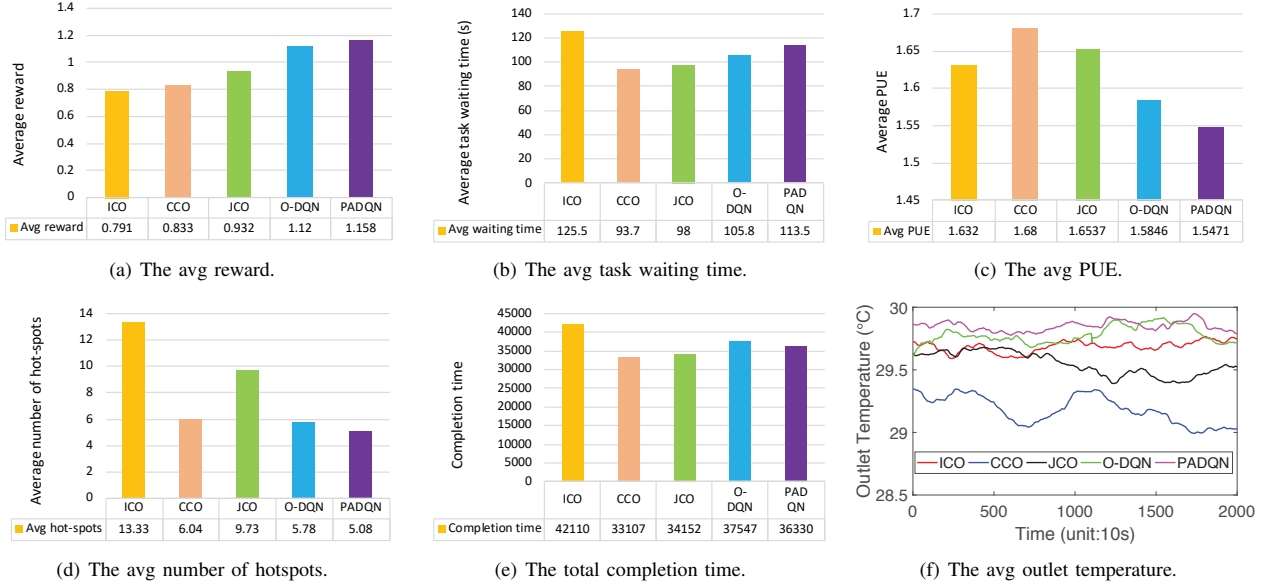


Fig. 4: The comparison results in terms of average reward, average task waiting time, average PUE, average number of hotspots, total completion time and average outlet temperature.

E. Performance Comparison

In order to verify the performance improvement of the proposed PADQN, we compare PADQN against the ICO, CCO, JCO and O-DQN algorithms described in Section V-B. The basic settings for these five algorithms are identical. The comparison results are illustrated in Fig. 4 and Fig. 5.

First, from Fig. 4(a), 4(b), 4(c), 4(d), 4(e), we can find that our proposed PADQN outperforms the other four algorithms in average reward, average PUE, the average number of hot-spots. Although the average task waiting time and the total completion time of PADQN may be worse than those of the other algorithms, the differences are very small. The proposed PADQN can derive a better energy efficiency than the baseline algorithms while achieving a good trade-off between saving energy and guaranteeing service quality (task waiting time). The root cause is that our proposed PADQN can catch the system dynamics better and coordinate both IT and cooling system more efficiently. It should be noted that the ICO algorithm has the worst average task waiting time, average number of hot-spots and total completion time because it has a fixed cooling constraint.

Second, we collect rack outlet temperatures and depict the temperature curves in Fig. 4(f). It can be observed that PADQN can keep a relatively higher but stable average outlet temperature (close to 30°C) compared to the baseline algorithm. The reason is that PADQN simultaneously considers the states of IT system as well as cooling system and uses the parameterized action space to achieve a more fine-grained control. The values are close to 30°C because the outlet temperature threshold $\psi_T = 30^\circ\text{C}$ for all algorithms if applicable. In addition, it can be observed that CCO algorithm is the worst algorithm in controlling the outlet temperature. It is

because that CCO algorithm only control the cooling power according to the workload and the cooling power is often over-provisioning for safe operation. That is also the reason why the CCO algorithm has the worst PUE.

Third, we also compare the proposed PADQN with the baseline algorithms in terms of energy consumption as illustrated in Fig. 5. To evaluate these algorithms, we use 2,000 tasks to initialize the data center and use another 2,000 tasks for evaluation. From Fig. 5(a), it can be observed that the power consumptions of both PADQN and O-DQN are largely less than that of other baseline algorithms before time 500. This is because the PADQN and O-DQN will consider whether or not allocating a new job degrades the long-term performance of the data center (e.g. generating more hot-spots, decreasing PUE), while the other baseline algorithms will dispatch a job to a server only according to an immediate reward. In addition, we can find that PADQN is more stable than O-DQN on IT power consumption (around time 600 – time 1300). The potential reason is that PADQN makes a more accurate control for the cooling equipment with the parameterized action space than O-DQN. A discrete action applied by O-DQN may lead to over-provisioning on cold air. This over-provisioning enables O-DQN to allocate more tasks to the data center than PADQN. Therefore, O-DQN consumes more energy during time 600–1300 in Fig. 5(a). Also, it can be observed in Fig. 5(b) that the growth rate of the cooling power consumption of O-DQN is faster than that of PADQN, which just means that O-DQN provides more cold air during that time. According to the data in Fig. 5, we can calculate that the proposed PADQN can save up to 7%, 15%, 10% and 5% energy compared to ICO, CCO, JCO and O-DQN respectively.

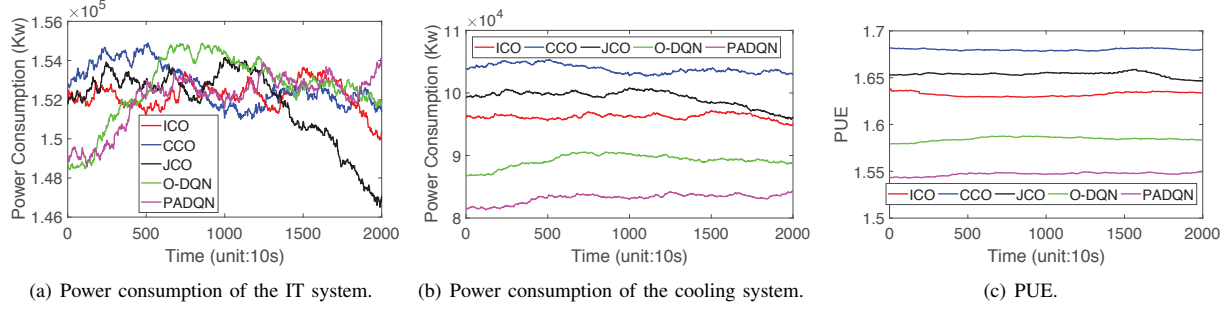


Fig. 5: The comparison results in power consumption. According to these results, we can calculate that the proposed PADQN can save up to 7%, 15%, 10% and 5% energy compared to ICO, CCO, JCO and O-DQN respectively.

F. Performance sensitivity with respect to t_{cool}

We conduct further experiments to investigate the effect of the decision interval t_{cool} for the cooling system. As discussed before, we assume that 1) the default decision interval is d seconds and 2) we need to make a decision at every decision epoch for the IT system and at every t_{cool} decision epochs for the cooling system. In our above experiments, we set $t_{cool} = 30$ which means that we make a decision for the cooling system every 5 minutes. In this section, we carry out several experiments by setting t_{cool} to 1, 30 and 90 (i.e., decision intervals are every 10 seconds, 5 minutes and 15 minutes respectively). $t_{cool} = 1$ implies the IT system and cooling system will be controlled concurrently. The other settings are the same as those in Section V-A.

TABLE I: The effect of the decision interval t_{cool}

Metric	1	30	90
Average reward	1.103	1.154	1.147
Average task waiting time	124.0s	111.5s	130.9s
Average PUE	1.5911	1.5471	1.6175
Average number of hot-spots	6.58	5.58	8.38
Total completion time	39547s	37547s	41152s

It can be observed from Table I that the performance with $t_{cool} = 30$ is better than that with $t_{cool} = 1$ and $t_{cool} = 90$. When $t_{cool} = 1$, it means the two-time-scale control is not applied, and the cooling equipment will be adjusted at each epoch. Due to the thermal lag, there will not be any obvious changes on thermal in a very short time duration (like 10 seconds). Therefore, the airflow rate would be continually and cumulatively increased (or reduced) when the DRL agent see a hot-spot (or a point where the temperature is too low). In this case, 1) the cooling equipment will be adjusted frequently which may lead to a frequent fluctuation in cold air provisioning, and 2) over-provisioning or under-provisioning may happen and degrade the performance (e.g. resulting in more hot-spots). When $t_{cool} = 90$, the cooling system is adjusted at every 15 minutes. This control with such a low frequency for cooling system cannot make a quick response to the workload changes as well as the thermal changes, so the performance with $t_{cool} = 90$ is also unsatisfactory.

VI. CONCLUSION

In this paper, we have investigated the problem of joint optimization of task scheduling and cooling control for energy efficient data centers. A DRL-based optimization framework, named DeepEE, is designed and implemented to improve the energy efficiency for a data center by considering the IT and cooling systems concurrently. In DeepEE, PADQN algorithm is proposed to jointly optimize the job scheduling for the IT system and the airflow rate control for the cooling system. PADQN employs parameterized action space technique to solve the hybrid action space problem and uses a two-time-scale control method to coordinate the IT and cooling systems more accurately and efficiently. The extensive simulation experiments on our developed simulation platform have shown that 1) the proposed PADQN can save up to 15% and 10% energy respectively compared to the baseline siloed and joint optimization algorithms respectively; 2) PADQN is more stable on power consumption than the baseline algorithms due to using the parameterized action space. 3) PADQN can achieve a better trade-off between saving energy and guaranteeing service quality. In the future, we plan to use event-based optimization methods to solve the issues of diverse time constants better.

ACKNOWLEDGMENT

This work is funded by National Research Foundation (NRF) via the Green Data Centre Research (GDCR) and Green Buildings Innovation Cluster (GBIC), administered by Informations Media Development Authority (IMDA) and Building and Construction Authority (BCA) respectively.

REFERENCES

- [1] W. Zhang, Y. Wen, Y. W. Wong, K. C. Toh, and C. Chen, "Towards joint optimization over ICT and cooling systems in data centre: A Survey," *IEEE Communications Surveys Tutorials*, vol. 18, no. 3, pp. 1596–1616, 2016.
- [2] J. Meng, E. Llamas *et al.*, "Communication and cooling aware job allocation in data centers for communication-intensive workloads," *Journal of Parallel and Distributed Computing*, vol. 96, pp. 181 – 193, 2016.

- [3] A. Beghi, L. Cecchinato *et al.*, “Modelling and control of a free cooling system for data centers,” *Energy Procedia*, vol. 140, pp. 447–457, 2017.
- [4] Y. Li, Y. Wen, K. Guan, and D. Tao, “Transforming cooling optimization for green data center via deep reinforcement learning,” *arXiv preprint arXiv:1709.05077*, 2017.
- [5] J. Wan, X. Gui, R. Zhang, and L. Fu, “Joint cooling and server control in data centers: A cross-layer framework for holistic energy minimization,” *IEEE Systems Journal*, vol. 12, no. 3, pp. 2461–2472, Sept 2018.
- [6] H. Sun, P. Stolf, and J.-M. Pierson, “Spatio-temporal thermal-aware scheduling for homogeneous high-performance computing datacenters,” *Future Generation Computer Systems*, vol. 71, pp. 157 – 170, 2017.
- [7] M. Polverini, A. Cianfrani, S. Ren, and A. V. Vasilakos, “Thermal-aware scheduling of batch jobs in geographically distributed data centers,” *IEEE Transactions on Cloud Computing*, vol. 2, no. 1, pp. 71–84, Jan 2014.
- [8] R. Zhou, Z. Wang, C. E. Bash, A. McReynolds *et al.*, “A holistic and optimal approach for data center cooling management,” in *American Control Conference (ACC)*, pp. 1346–1351, 2011.
- [9] L. Parolini, B. Sinopoli, B. H. Krogh, and Z. Wang, “A cyberphysical systems approach to data center modeling and control for energy efficiency,” *Proceedings of the IEEE*, vol. 100, no. 1, pp. 254–268, Jan 2012.
- [10] Q. Fang, J. Wang, Q. Gong, and M. Song, “Thermal-aware energy management of an hpc data center via two-time-scale control,” *IEEE Transactions on Industrial Informatics*, vol. 13, no. 5, pp. 2260–2269, Oct 2017.
- [11] V. Mnih, K. Kavukcuoglu, D. Silver *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, p. 529, 2015.
- [12] “6SigmaDCX.” [Online]. Available: <https://www.futurefacilities.com>
- [13] Q. Tang, S. K. S. Gupta, and G. Varsamopoulos, “Energy-efficient thermal-aware task scheduling for homogeneous high-performance computing data centers: A cyber-physical approach,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 19, no. 11, pp. 1458–1472, Nov 2008.
- [14] A. Chavan, M. I. Alghamdi, X. Jiang *et al.*, “Tiger: Thermal-aware file assignment in storage clusters,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 2, pp. 558–573, Feb 2016.
- [15] A. Sansottera and P. Cremonesi, “Cooling-aware workload placement with performance constraints,” *Performance Evaluation*, vol. 68, no. 11, pp. 1232 – 1246, 2011, special Issue: Performance 2011.
- [16] L. Wang, S. U. Khan, and J. Dayal, “Thermal aware workload placement with task-temperature profiles in a data center,” *The Journal of Supercomputing*, vol. 61, no. 3, pp. 780–803, 2012.
- [17] Z. Wang, C. Bash, N. Tolia *et al.*, “Optimal fan speed control for thermal management of servers,” in *ASME 2009 InterPACK Conference*, pp. 709–719. American Society of Mechanical Engineers, 2009.
- [18] E. Pakbaznia and M. Pedram, “Minimizing data center cooling and server power costs,” in *Proceedings of the 2009 ACM/IEEE International Symposium on Low Power Electronics and Design*, pp. 145–150. New York, NY, USA: ACM, 2009.
- [19] “PBS Pro.” [Online]. Available: www.pbsworks.com
- [20] A. Banerjee, T. Mukherjee, G. Varsamopoulos, and S. K. Gupta, “Integrating cooling awareness with thermal aware workload placement for hpc data centers,” *Sustainable Computing: Informatics and Systems*, vol. 1, no. 2, pp. 134–150, 2011.
- [21] T. Mukherjee, A. Banerjee, G. Varsamopoulos, S. K. Gupta, and S. Rungta, “Spatio-temporal thermal-aware job scheduling to minimize energy consumption in virtualized heterogeneous data centers,” *Computer Networks*, vol. 53, no. 17, pp. 2888–2904, 2009.
- [22] “Parallel Workloads Archive.” [Online]. Available: <http://www.cs.huji.ac.il/labs/parallel/workload/logs.html>
- [23] F. Kong and X. Liu, “A survey on green-energy-aware power management for datacenters,” *ACM Computing Surveys (CSUR)*, vol. 47, no. 2, p. 30, 2015.
- [24] X. Fan, W.-D. Weber, and L. A. Barroso, “Power provisioning for a warehouse-sized computer,” in *ACM SIGARCH computer architecture news*, vol. 35, no. 2, pp. 13–23. ACM, 2007.
- [25] “Types of electrical meters in data centers.” [Online]. Available: <http://www.mcrinc.com/Documents/>
- [26] A. Albanna and H. Yousefi’zadeh, “Learning-constrained enhancement of cellular networks capacity,” *IEEE Transactions on Mobile Computing*, vol. 18, no. 1, pp. 153–164, Jan 2019.
- [27] W. Wang, Y. Yan, L. Zhang, R. Hong, and N. Sebe, “Collaborative sparse coding for multiview action recognition,” *IEEE MultiMedia*, vol. 23, no. 4, pp. 80–87, Oct 2016.
- [28] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” *arXiv preprint arXiv:1509.02971*, 2015.
- [29] W. Masson, P. Ranchod, and G. Konidaris, “Reinforcement learning with parameterized actions,” in *AAAI*, pp. 1934–1940, 2016.